

Prototype Design Pattern

1. Definition

The **Prototype Pattern** is a **creational design pattern** that lets you create new objects by **cloning existing ones** (prototypes), instead of constructing them from scratch.

It is especially useful when object creation is expensive, or when you want many copies of similar objects.

2. Why Prototype Came (The Need)

- Object creation can be **expensive** (complex initialization, heavy resource allocation, database/network calls).
 - Often you need **multiple similar objects** with minor differences.
 - Instead of repeatedly using `new`, cloning allows **reuse of an existing configured object**.
 - Ensures flexibility when **class type is unknown at runtime** but cloning is possible.
-

3. Key Idea

- Each object implements a `clone()` method (deep or shallow copy).
 - A **Prototype Registry** can store frequently used prototypes for easy duplication.
-

4. Code Examples

Java Example

```
// Prototype interface
interface Prototype extends Cloneable {
    Prototype clone();
}

// Concrete prototype
class Shape implements Prototype {
    private String type;
    private int x, y;

    public Shape(String type, int x, int y) {
        this.type = type;
        this.x = x;
        this.y = y;
    }
}
```

```

@Override
public Prototype clone() {
    return new Shape(this.type, this.x, this.y); // deep copy
}

public void draw() {
    System.out.println("Drawing " + type + " at (" + x + "," + y + ")");
}
}

public class PrototypeDemo {
    public static void main(String[] args) {
        Shape circle = new Shape("Circle", 10, 20);
        Shape circleCopy = (Shape) circle.clone();

        circle.draw();
        circleCopy.draw();
    }
}

```

Python Example

```

import copy

class Prototype:
    def clone(self):
        return copy.deepcopy(self)

class Document(Prototype):
    def __init__(self, name, content):
        self.name = name
        self.content = content

    def show(self):
        print(f"Document: {self.name}, Content: {self.content}")

# Usage
doc1 = Document("Report", "This is the original content")
doc2 = doc1.clone()
doc2.name = "Report Copy"

doc1.show()
doc2.show()

```

5. Advantages

- ✓ Faster object creation (avoids costly initialization)
- ✓ Useful when object construction is complex

- ✓ Simplifies code when many similar objects are needed
 - ✓ Works when exact class isn't known at runtime but cloning is supported
-

6. Applications

-  **Game Development** → Cloning enemies, power-ups, bullets.
 -  **Graphic Editors** → Copying shapes, images, UI components.
 -  **Document Editors** → Duplicate templates, styles, predefined content.
 -  **Database/Resource Objects** → Clone heavy DB connections, configs.
 -  **AI/Simulations** → Copy entities with small variations.
-

7. When *Not* to Use

- ✗ If objects are **simple** (better to use `new`).
 - ✗ If cloning requires **deep copy of complex references** (hard to maintain).
 - ✗ If class structure is **unstable** (frequent changes → clone logic breaks).
 - ✗ For **immutable objects** (no need to clone, reuse directly).
-

8. Comparison with Other Creational Patterns

- **Factory / Abstract Factory** → Create **new** objects from scratch.
 - **Builder** → Step-by-step construction of complex objects.
 - **Prototype** → **Clone existing objects** instead of creating anew.
-

In Short

Use **Prototype** when you want to **duplicate existing objects efficiently** rather than re-create them from scratch.

Avoid it when objects are simple, immutable, or when deep copy becomes too complex.